

Simulation setup and analysis/post-processing

Akshay Patil, Clara García-Sánchez

3D Geoinformation Research Group, Faculty of Architecture and the Built Environment, Delft University
of Technology, The Netherlands

Agenda

- ❑ 1. Boundary Conditions for ABL flows
- ❑ 2. Numerical Solvers & Schemes
- ❑ 3. Consistency, Stability, Convergence & Accuracy
- ❑ 6. Dispersion set-up
- ❑ 7. Tree set-up

Boundary Conditions for ABL flows

Boundary Conditions for ABL flows

Physical models - equations

Assumptions

- ❑ Vertical velocity is zero
- ❑ Pressure is constant
- ❑ Shear stress is constant

$$(\mu_l + \mu_t) \frac{dU}{dz} = \tau_0 = \rho u_*^2 \quad \mu_t = \rho C_\mu \frac{k^2}{\epsilon}$$

- ❑ k and ϵ satisfy their conservation equations:

$$\begin{aligned} \frac{\partial}{\partial z} \left(\frac{\mu_t}{\sigma_k} \frac{\partial k}{\partial z} \right) + \mu_t \left(\frac{\partial U}{\partial z} \right)^2 - \rho \epsilon &= 0 \\ \frac{\partial}{\partial z} \left(\frac{\mu_t}{\sigma_\epsilon} \frac{\partial \epsilon}{\partial z} \right) + C_1 \mu_t \left(\frac{\partial U}{\partial z} \right)^2 \frac{\epsilon}{k} - C_2 \rho \frac{\epsilon^2}{k} &= 0 \end{aligned}$$

Boundary Conditions for ABL flows

ABL Logarithmic Inflow

Using the previous assumptions, the following initial and inflow profiles are derived:

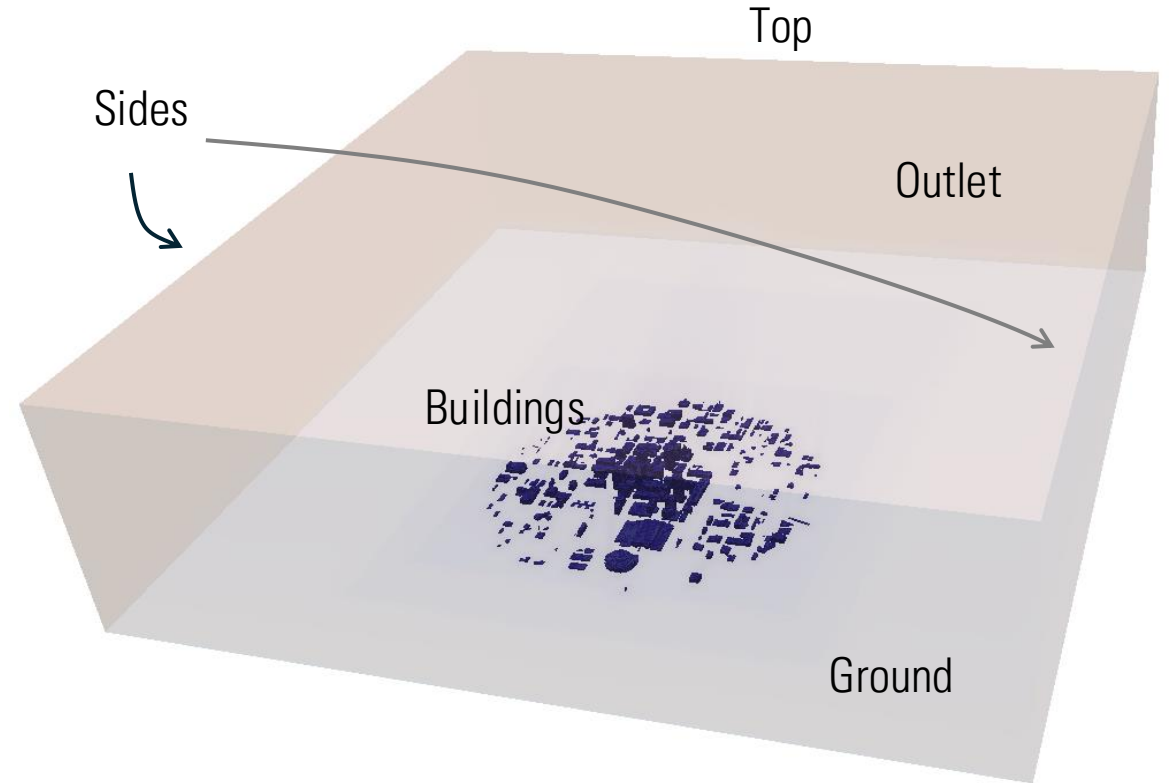
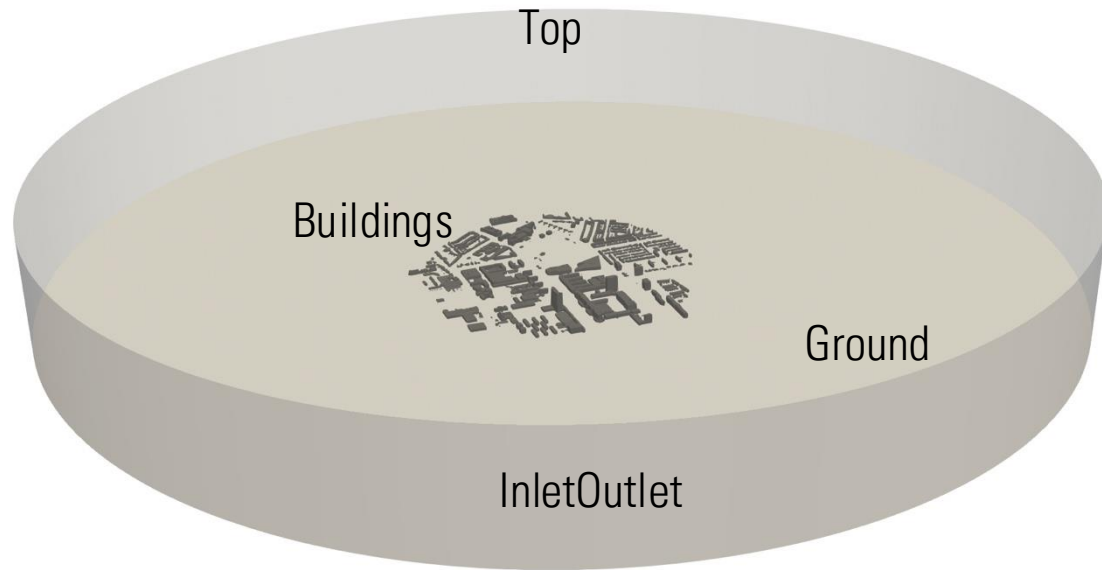
$$U(z) = \frac{u_*}{\kappa} \log \left(\frac{z + z_0}{z_0} \right)$$

$$k(z) = \frac{u_*^2}{\sqrt{C_\mu}}$$

$$\varepsilon(z) = \frac{u_*^3}{\kappa (z + z_0)}$$

Boundary Conditions for ABL flows

Boundary conditions within the domain



Boundary Conditions for ABL flows

Boundary conditions within the domain

Outflow

The top boundary condition is very important for sustaining equilibrium boundary layer profiles. Two options:

1. Symmetry conditions to ensure the profiles to develop naturally
2. Another option is to prescribe the values for the velocities and the turbulence quantities of the inflow profile at the height of the top boundary over the entire top boundary (Blocken et al., 2006).

Sides

At lateral boundaries symmetry boundary conditions are frequently used when the approach flow direction is parallel to them. As symmetry boundary conditions enforce a parallel flow by requiring a vanishing normal velocity component at the boundary, the boundary should be positioned far enough away from the built area of interest to not lead to an artificial acceleration of the flow in the region of interest

In the case of multi-directional runs (round domain) the lateral boundaries become inflow and outflow boundaries with corresponding boundary conditions

Boundary Conditions for ABL flows

Boundary conditions within the domain

Top

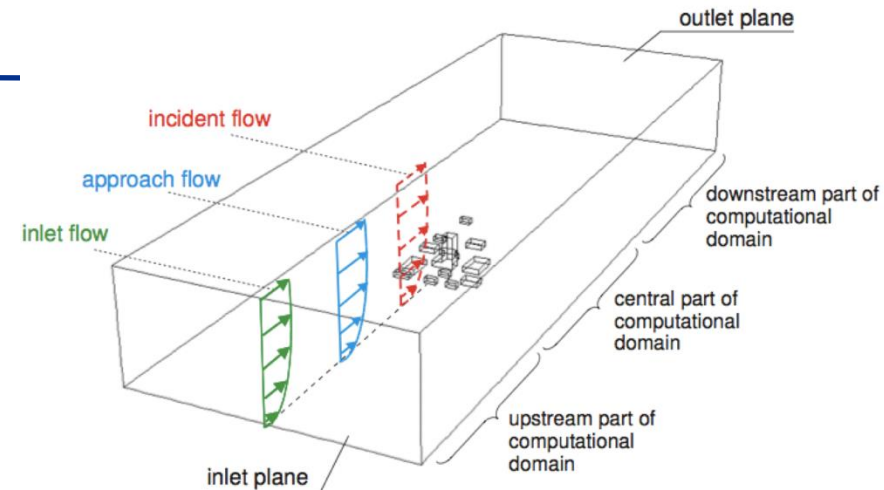
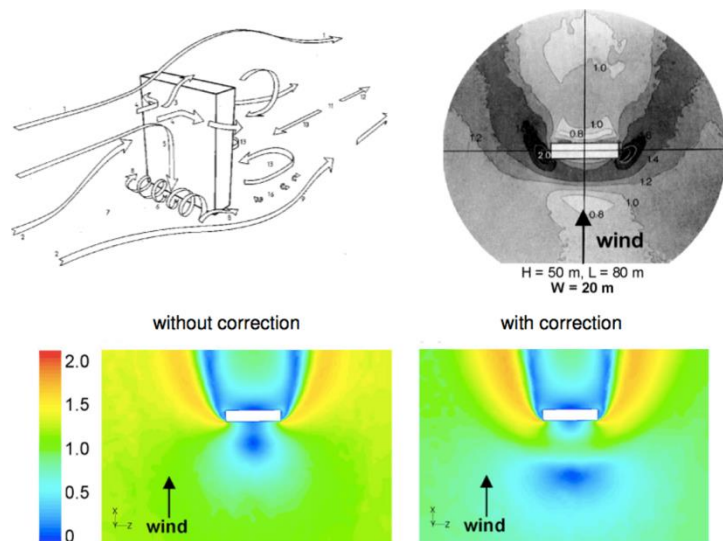
- ❑ At the boundary behind the obstacles (where all or most of the fluid leaves the computational domain), open boundary conditions are used in commercial CFD and microscale obstacle-accommodating meteorology models.
- ❑ The open boundary conditions in commercial CFD codes are either outflow or constant static pressure boundary conditions (pressure = atmospheric conditions or 0 if relative pressure).
- ❑ With an outflow boundary condition the derivatives of all flow variables are forced to vanish, corresponding to a fully developed flow. Therefore this boundary should be ideally far enough away from the built area to not have any fluid entering into the computational domain through this boundary.
- ❑ This also holds for the use of a constant static pressure at the outflow boundary, where then only the derivatives of all other flow variables are forced to vanish.

Boundary Conditions for ABL flows

Boundary conditions within the domain

Ground – wall Function

- ❑ At the ground velocity value vanishes, so normally no slip or fixed value 0 are used
- ❑ For pressure, zero gradient conditions are assumed
- ❑ For terrain and semantic surfaces (water, vegetation) wall functions with roughness characteristics are applied to ensure profile homogeneity along the inflow direction



Blocken B, Stathopoulos T, Carmeliet J. 2007. CFD simulation of the Atmospheric Boundary Layer – wall function problems. *Atmospheric Environment* 41(2): 238-252.

Boundary Conditions for ABL flows

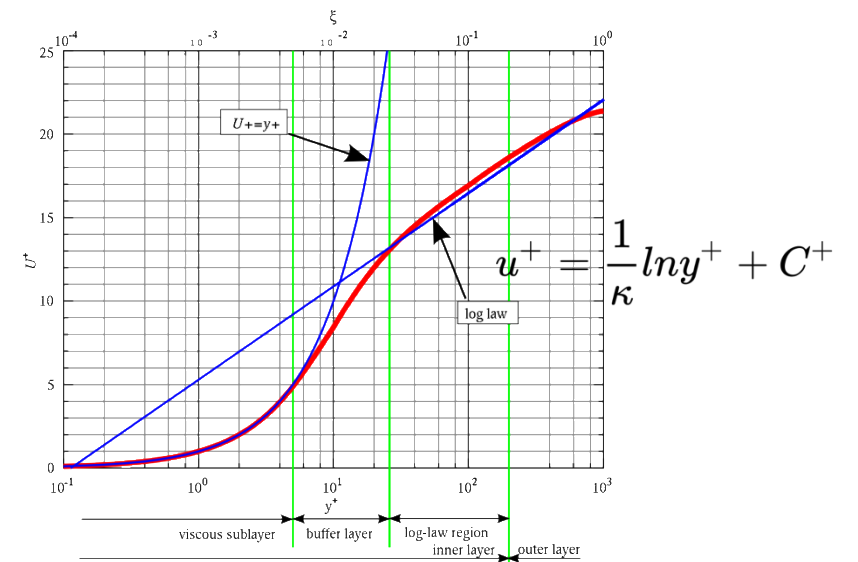
Boundary conditions within the domain

Ground – wall Function

With commercial codes two options exist:

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)
2. Implement a wall function based on the aerodynamic roughness in the solve (Parente 2011)

What is a wall function?



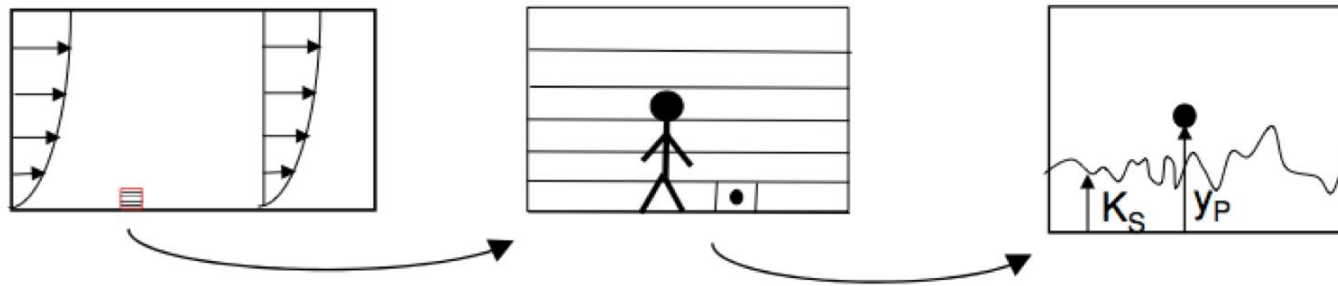
[<http://brennen.caltech.edu/fluidbook/basicfluidynamics/turbulence/lawofthewall.pdf>]

Boundary Conditions for ABL flows

Boundary conditions within the domain

Ground – wall Function – option 1

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)
 - ❑ High mesh resolution in the vertical direction close to the ground: 3-4 cells below 1.5m (COS732)
 - ❑ Distance from the center of the wall-adjacent cell to the wall larger than the physical roughness height or “equivalent sand-grain roughness” ($y_p > k_s$)
 - ❑ Avoid unintended streetwise gradients by using the correct relationship between k_s and y_p



Boundary Conditions for ABL flows

Boundary conditions within the domain

Ground – wall Function – option 1

1. Modify a constant in existing (rough) wall functions to match the friction velocity to the correct value (Blocken 1997)

Solution

1. Determine y_0
2. Select y_p (as large as possible, but with sufficient resolution)
3. Determine $k_{s,ABL}$ with the above equation for $C_s=0.5$
4. If $k_{s,ABL} < y_p$, set $k_{s,ABL} = y_p$
5. Determine updated C_s from the above equation

$$k_{s,ABL} = \frac{9.793 y_0}{C_s}$$

Boundary Conditions for ABL flows

Boundary conditions within the domain

Ground – wall Function – option 2

2. Implement a wall function based on the aerodynamic roughness in the solve (Parente 2011)

Richards and Hoxey (1993)

$$U_p = \frac{u_*}{\kappa} \ln \left(\frac{z + z_0}{z_0} \right)$$
$$\epsilon_p = \frac{C_\mu^{0.75} k^{1.5}}{\kappa (z + z_0)}$$
$$G_k = \frac{\tau_w^2}{\rho \kappa C_\mu^{0.25} k^{0.5} (z + z_0)}$$

Implementation
(Fluent, OpenFOAM, Fine Open)

$$\frac{U_p}{u_*} = \frac{1}{\kappa} \ln \left(\tilde{E} \tilde{y}^+ \right)$$

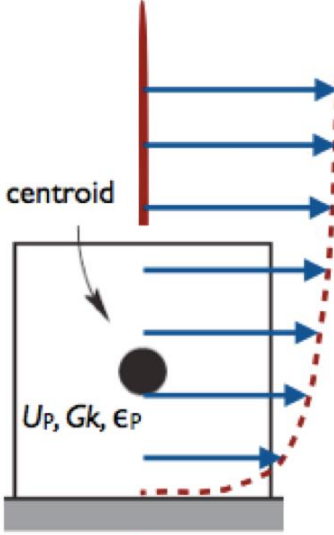
Smooth wall

$$\tilde{y}^+ = y^+ \quad \tilde{E} = E$$

Rough wall

$$\tilde{y}^+ = \frac{u_* (z + z_0)}{\nu} \quad \tilde{E} = \frac{\nu}{z_0 u_*}$$

Switch based on local surface properties



Boundary Conditions for ABL flows

Boundary conditions within the domain

Ground – wall Function – option 2

2. Implement a wall function based on the aerodynamic roughness in the solve (Parente 2011)

$$U = \frac{u_*}{\kappa} \ln \left(\frac{z + z_0}{z_0} \right) \quad k = \frac{u_*^2}{\sqrt{C_\mu}} \quad \epsilon = \frac{u_*^3}{\kappa (z + z_0)}$$
$$\sigma_\epsilon = \frac{\kappa^2}{(C_{\epsilon 2} - C_{\epsilon 1}) \sqrt{C_\mu}} \quad \text{or} \quad S_\epsilon(z) = \frac{\rho u_*^4}{(z + z_0)^2} \left(\frac{(C_{\epsilon 2} - C_{\epsilon 1}) \sqrt{C_\mu}}{\kappa^2} - \frac{1}{\sigma_\epsilon} \right)$$

Some issues with this approach:

1. Default value of $C_\mu=0.09$ often results in too low TKE compared to experimental data —> we can modify its value to adjust it to k ; adjust σ_ϵ or introduce source term S_ϵ accordingly
2. k profile can vary with height —> modify k and ϵ solutions
3. Not adapted for other than k -epsilon standard turbulence model

Boundary Conditions for ABL flows

Boundary conditions within the domain

Ground – wall Function – option 2

	y_0 (m)	Landscape description
1	0.0002 Sea	Open sea or lake (irrespective of the wave size), tidal flat, snow-covered flat plain, featureless desert, tarmac, concrete, with a free fetch of several kilometres.
2	0.005 Smooth	Featureless land surface without any noticeable obstacles and with negligible vegetation; e.g. beaches, pack ice without large ridges, morass, and snow-covered or fallow open country.
3	0.03 Open	Level country with low vegetation (e.g. grass) and isolated obstacles with separations of at least 50 obstacle heights; e.g. grazing land without windbreaks, heather, moor and tundra, runway area of airports.
4	0.10 Roughly open	Cultivated area with regular cover of low crops, or moderately open country with occasional obstacles (e.g. low hedges, single rows of trees, isolated farms) at relative horizontal distances of at least 20 obstacle heights.
5	0.25 Rough	Recently-developed “young” landscape with high crops or crops of varying height, and scattered obstacles (e.g. dense shelterbelts, vineyards) at relative distances of about 15 obstacle heights.
6	0.50 Very rough	“Old” cultivated landscape with many rather large obstacle groups (large farms, clumps of forest) separated by open spaces of about 10 obstacle heights. Also low large vegetation with small interspaces such as bush land, orchards, young densely-planted forest.
7	1.0 Closed	Landscape totally and quite regularly covered with similar-size large obstacles, with open spaces comparable to the obstacle heights; e.g. mature regular forests, homogeneous cities or villages.
8	≥ 2.0 Chaotic	Centres of large towns with mixture of low-rise and high-rise buildings. Also irregular large forests with many clearings.

Boundary Conditions for ABL flows

Boundary conditions within the domain

Round domain

	U	k	ε	ν_t	p
InletOutlet	ABLinletVelocity	ABLinletk	ABLinletEpsilon	calculated	freeStream
Top	symmetry	symmetry	symmetry	symmetry	symmetry
Ground/Terrain	fixedValue 0	kqRWallFunction	atmEpsilonWallFunction (with z0 per class)	atmNutmWallFunction (with z0 per class)	zeroGradient
Buildings	fixedValue 0	kqRWallFunction	epsilonWallFunction	nutkWallFunction	zeroGradient
Vegetation/Water	fixedValue 0	kqRWallFunction	atmEpsilonWallFunction (with z0 per class)	atmNutmWallFunction (with z0 per class)	zeroGradient

Boundary Conditions for ABL flows

Boundary conditions within the domain

Rectangular domain

	U	k	ε	ν_t	p
Inlet	ABLinletVelocity	ABLinletk	ABLinletEpsilon	calculated	zeroGradient
Outlet	zeroGradient	zeroGradient	zeroGradient	calculated	fixedValue 0
Top/Sides	symmetry	symmetry	symmetry	symmetry	symmetry
Ground/Terrain	fixedValue 0	kqRWallFunction	atmEpsilonWallFunction (with z0 per class)	atmNutmWallFunction (with z0 per class)	zeroGradient
Buildings	fixedValue 0	kqRWallFunction	epsilonWallFunction	nutkWallFunction	zeroGradient
Vegetation/Water	fixedValue 0	kqRWallFunction	atmEpsilonWallFunction (with z0 per class)	atmNutmWallFunction (with z0 per class)	zeroGradient

Questions so far?

Numerical Solvers & Schemes

Numerical Solvers

Introduction

After discretization you end up with an algebraic system of matrices that will be solved iteratively using a computer system:

$$\begin{matrix} & & \textcolor{red}{A} & & & & \textcolor{red}{\phi} & & \textcolor{red}{B} \\ \begin{bmatrix} A_{11} & A_{12} & 0 & \dots & 0 \\ A_{21} & A_{22} & A_{23} & \dots & 0 \\ 0 & A_{32} & A_{33} & \dots & A_{3n} \\ \dots & \dots & \dots & \ddots & \dots \\ 0 & 0 & A_{n3} & \dots & A_{nn} \end{bmatrix} & \begin{bmatrix} \phi_1 \\ \phi_2 \\ \dots \\ \phi_i \\ \dots \\ \phi_{n-1} \\ \phi_{nn} \end{bmatrix} & = & \begin{bmatrix} B_1 \\ B_2 \\ \dots \\ B_i \\ \dots \\ B_{n-1} \\ B_{nn} \end{bmatrix} \end{matrix}$$

Numerical Solvers

Types of numerical solvers

System of linear or non-linear algebraic equations that needs to be solved by some numerical methods

Complexity and size of the system —> depends on dimensionality and geometry of the physical problem

Two families:

- ❑ Direct methods (simple 1D problems: Gaussian elimination, Cramer's rule, Thomas algorithm)
- ❑ **Iterative methods** (for non-linear problems they are necessary, but also valuable for sparse linear systems)

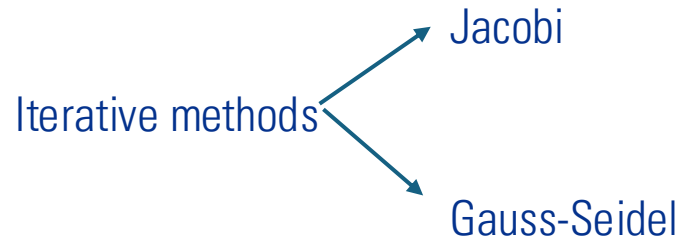
Numerical Solvers

Iterative methods

Unfortunately most CFD problems usually result in a large system of non-linear equations

The only option then are iterative methods: where one guesses the solution and uses the equation to systematically improve the solution until it reaches some level of convergence.

Iterative methods can be faster than direct methods, if the number of iterations is small



Numerical Solvers

Iterative methods – Jacobi

- ❑ Simplest
- ❑ Requires that all nodal variables (ϕ_j) non diagonal elements area assumed to be known at each iteration step k
- ❑ Nodal variables (ϕ_i) are treated as unknown at each iteration step $k+1$
- ❑ The iteration starts by an initial guessing of the nodal variables $\phi_j(k=0)$

$$A\phi = B \qquad \sum_{j=1}^{i-1} A_{ij} \phi_j + A_{ii} \phi_i + \sum_{j=i+1}^n A_{ij} \phi_j = B_i$$

Solving for ϕ_i

$$\phi_i^{k+1} = \frac{B_i}{A_{ii}} - \sum_{j=1}^{i-1} \frac{A_{ij}}{A_{ii}} \phi_j^{(k)} - \sum_{j=i+1}^n \frac{A_{ij}}{A_{ii}} \phi_j^{(k)}$$

Numerical Solvers

Iterative methods – Gauss Seidel

- ❑ Improves the Jacobi method
- ❑ The updated nodal variables $\phi_j^{(k+1)}$ are immediately used on the RHS of the previous equation
- ❑ 2 times faster than Jacobi

Solving for ϕ_i

$$\phi_i^{k+1} = \frac{B_i}{A_{ii}} - \sum_{j=1}^{i-1} \frac{A_{ij}}{A_{ii}} \phi_j^{(k+1)} - \sum_{j=i+1}^n \frac{A_{ij}}{A_{ii}} \phi_j^{(k)}$$

Numerical Solvers

Iterative methods – Convergence and relaxation

Convergence: We ensure that difference is below a very low value, the lower the value the more accurate the solution will be, but more iterations are needed.

$$\phi_j^{k+1} - \phi_j^k \ll 10^{-5}$$

Successive overrelaxation:

- ❑ The convergence to a solution can be enhanced by utilizing successive overrelaxation
- ❑ We use an extrapolation procedure where the intermediate nodal variables $\phi_j^{(k+1)}$ are further advanced by a weighted average of the current values of $\phi_j^{(k+1)}$ with the previous values of $\phi_j^{(k)}$:

$$\phi_j^{(k+1)} = (1 - \lambda)\phi_j^k + \lambda\phi_j^{(k+1)}$$

- ❑ Here λ is a relaxation factor whose value is normally found by trial and error
- ❑ It can make the Gauss-seidel method much faster choosing $1 < \lambda < 2$

Numerical Solvers

Pressure-velocity coupling

Because of the incompressible assumption, solution to the GE is complicated by the lack of an independent pressure equation

Momentum (3 equations) and continuity equation (1 equation) $\rightarrow$ draw a self-contained system since we have 4 variables: u, v, w, p .

Velocity



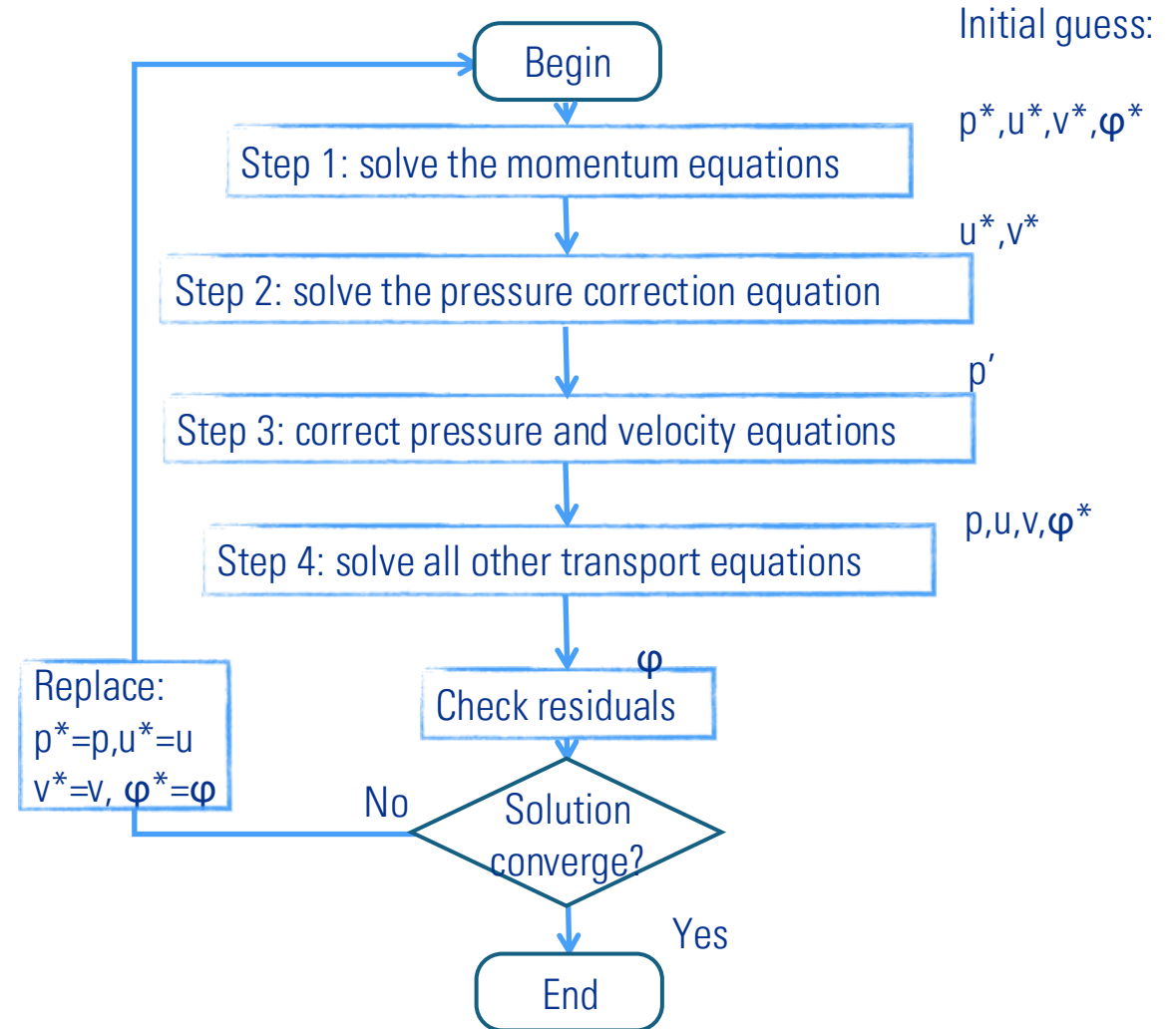
Pressure

Without pressure equation, we need a coupling between pressure and velocity

Numerical Solvers

Pressure-velocity coupling

SIMPLE scheme is a guess-and-correct process for calculating the pressure through the solution of a pressure correction equation



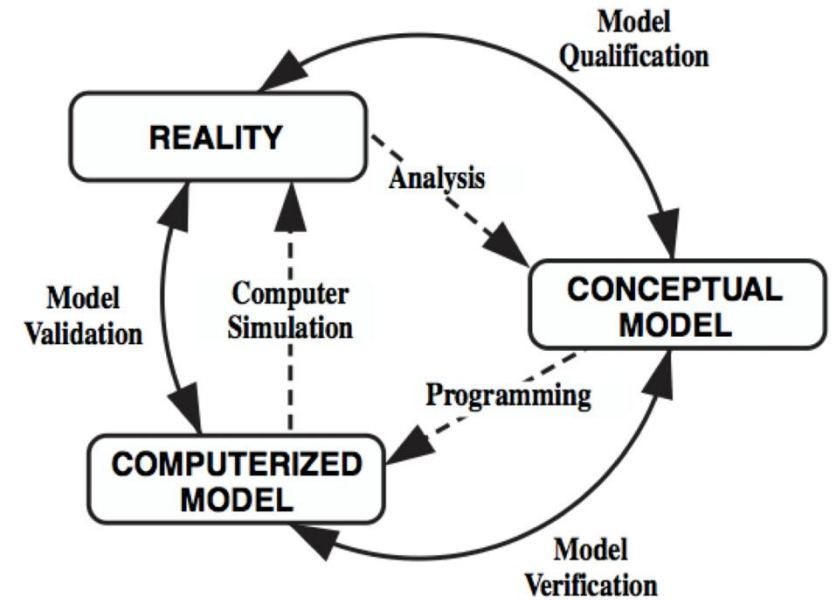
Schemes

Introduction

Qualification: is the conceptual model appropriate to provide acceptable level of agreement with the application? Are we solving the correct equations?

Verification: 1) model implementation accurately represents the conceptual model; 2) the solution of the model is accurate. Are we solving the equations correctly?

Validation: determine to which degree the model is an accurate representation of reality from the perspective of the intended application



Once the verification of the code is one, qualification == validation

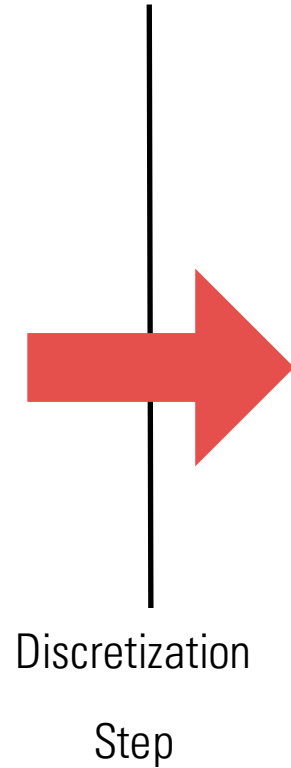
Schemes

Introduction

Mathematical formulation

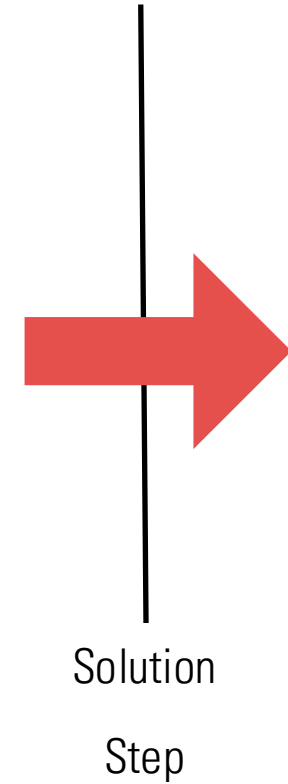
PDE

IC/BC



Numerical formulation

ADE



Numerical Solution

Schemes

Mathematical model

$$\frac{\partial \overline{U_j}}{\partial x_j} = 0 \qquad \frac{\partial \overline{U_i}}{\partial t} + \overline{U_j} \frac{\partial \overline{U_i}}{\partial x_j} = -\delta_{i3} g + f_c \varepsilon_{ij3} \overline{U_j} - \frac{1}{\overline{\rho}} \frac{\partial \overline{P}}{\partial x_i} + \frac{\partial}{\partial x_j} \left[\frac{(\mu + \mu_t)}{\rho} \frac{\partial \overline{U_i}}{\partial x_j} \right]$$

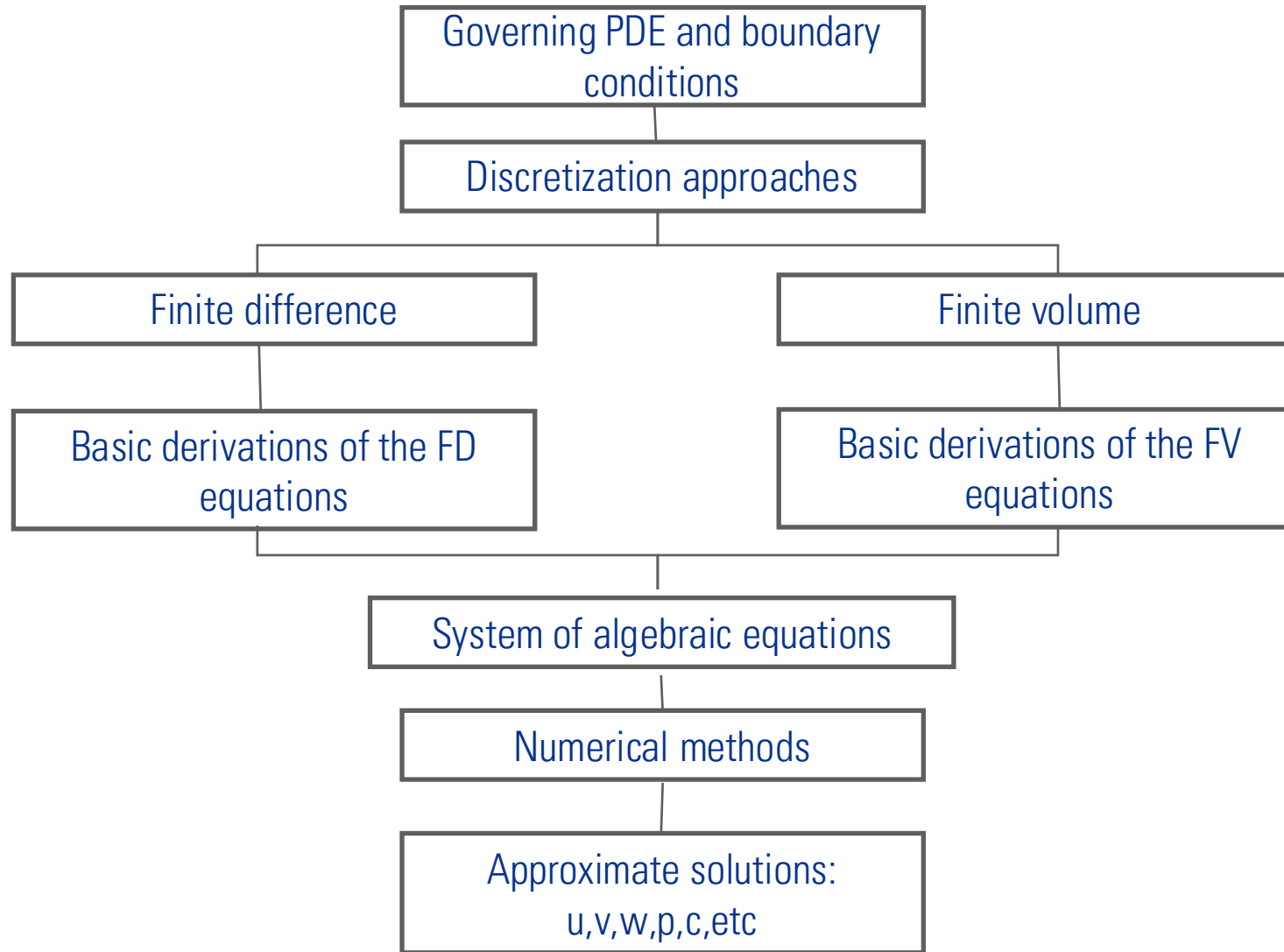
$$\overline{U_j} \frac{\partial k}{\partial x_j} = \frac{\partial}{\partial x_j} \left[\left(\frac{\mu + \mu_t}{\rho \sigma_k} \right) \frac{\partial k}{\partial x_j} \right] + P_k - \varepsilon$$

$$\overline{U_j} \frac{\partial \varepsilon}{\partial x_j} = \frac{\partial}{\partial x_j} \left[\left(\frac{\mu + \mu_t}{\rho \sigma_\varepsilon} \right) \frac{\partial \varepsilon}{\partial x_j} \right] + C_{\varepsilon 1} \frac{\varepsilon}{k} P_k - C_{\varepsilon 2} \frac{\varepsilon^2}{k}$$

$$\mu_t = \rho C_\mu \frac{k^2}{\varepsilon}$$

Schemes

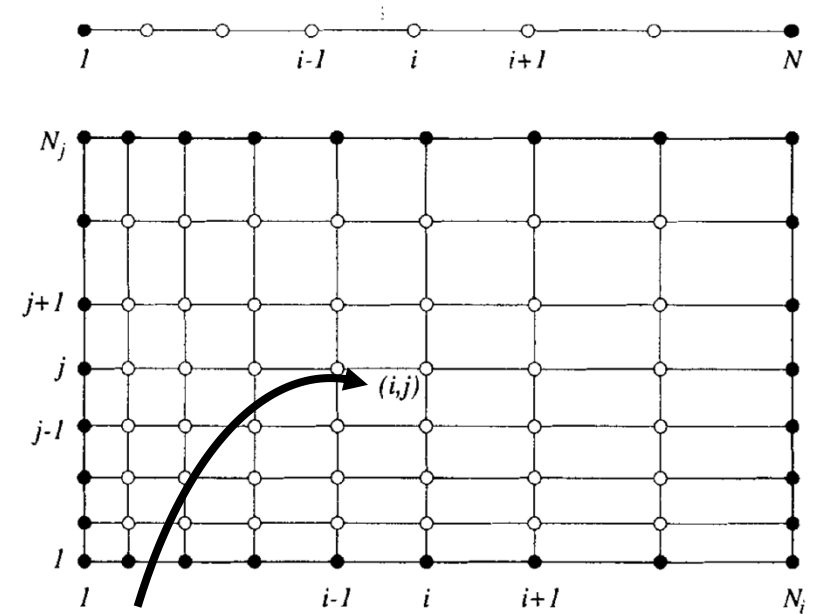
Discretization



Schemes

Discretization – Finite Differences

- ❑ The region of interest (where all the fluid we want to model is) is transformed in a grid
- ❑ The solution of the original equations is only computed at specific locations of the grid (normally center/faces)
- ❑ Commonly used for regular meshes
- ❑ Uses the differential form of the equations
- ❑ At each nodal point, Taylor series expansions are used to generate finite difference approximations to the partial derivatives of the governing equations
- ❑ Conservation property is not enforced



An unknown value of a field variable that depends on the neighbouring nodes, providing and algebraic equation

Simplest case since it is uniform spacing

If you are resolving in time, you will also discretise with respect to time

Schemes

Discretization – Finite Differences

- The continuous derivatives are transformed in discrete algebraic relationships between the grid point values (Finite Difference Methods)
- Using the derivative definition:

$$\frac{\partial \phi(x)}{\partial x} = \lim_{\Delta x \rightarrow 0} \frac{\phi(x + \Delta x) - \phi(x)}{\Delta x}$$

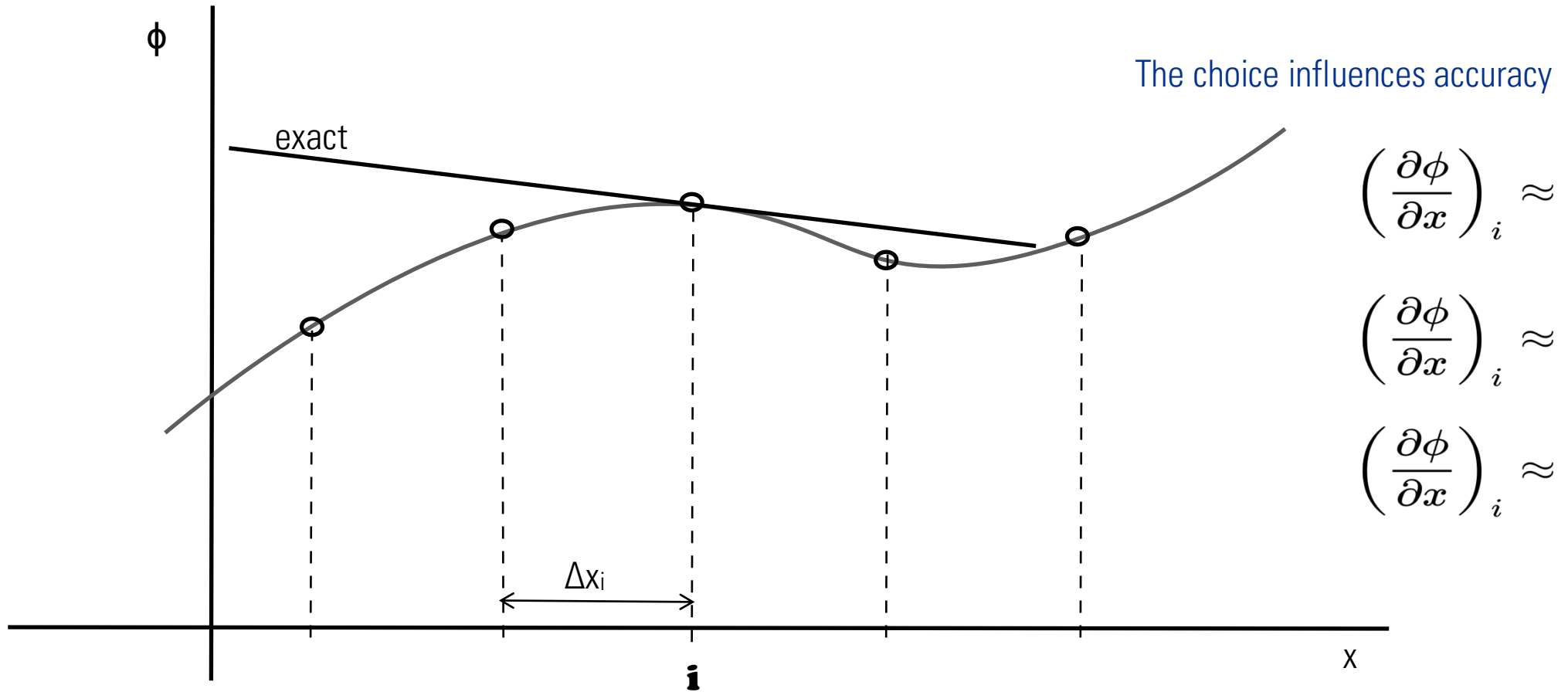
- But without evaluating the limit:

$$\frac{\partial \phi(x)}{\partial x} \approx \frac{\phi(x + \Delta x) - \phi(x)}{\Delta x} = \frac{\phi_{i+1} - \phi_i}{\Delta x}$$

Discretization errors

Schemes

Discretization – Finite Differences



Schemes

Discretization – Finite Differences

The choice influences accuracy of the approximation

$$\phi(x) = \phi(x_i) + (x - x_i) \frac{\partial \phi}{\partial x} \Big|_i + \frac{(x - x_i)^2}{2!} \frac{\partial^2 \phi}{\partial x^2} \Big|_i + \dots + \frac{(x - x_i)^n}{n!} \frac{\partial^n \phi}{\partial x^n} \Big|_i$$

Using a forward discretisation:

$$\frac{\partial \phi}{\partial x} \Big|_i =$$

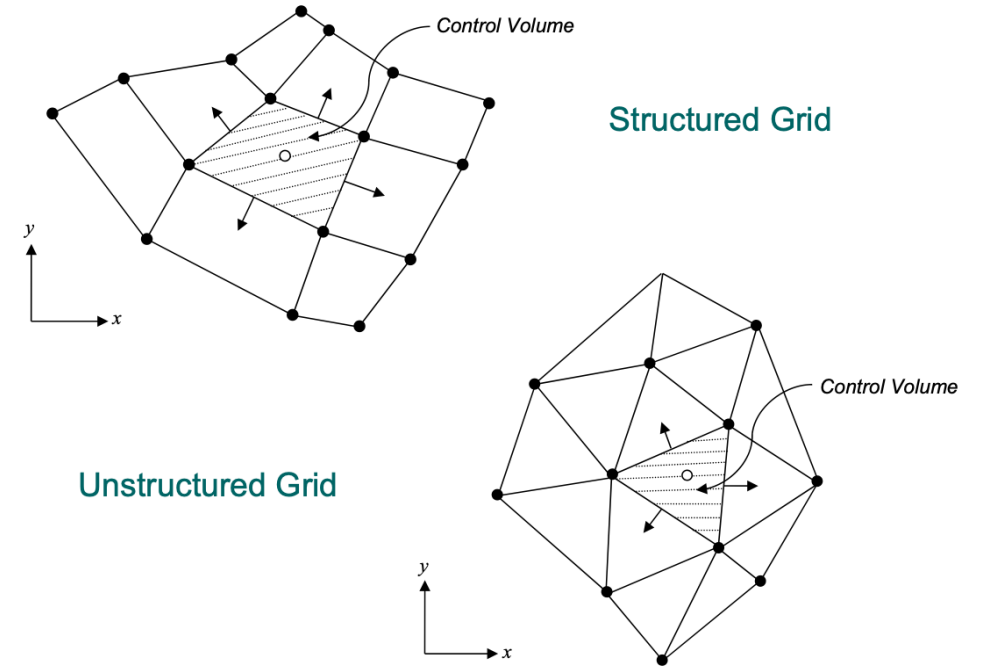
Using a central discretisation:

$$\frac{\partial \phi}{\partial x} \Big|_i =$$

Schemes

Discretization – Finite Volume

- ❑ The region of interest (where all the fluid we want to model is) is transformed in a set of contiguous control volumes
- ❑ The variables are computed at the centroids of each control volume
- ❑ Normally used for complex meshes
- ❑ Uses the integral form of the equations
- ❑ The grid defines the control volumes boundaries, the method is conservative so long as the surface integrals that area applied at these boundaries are the same as the control values sharing the boundary.



Schemes

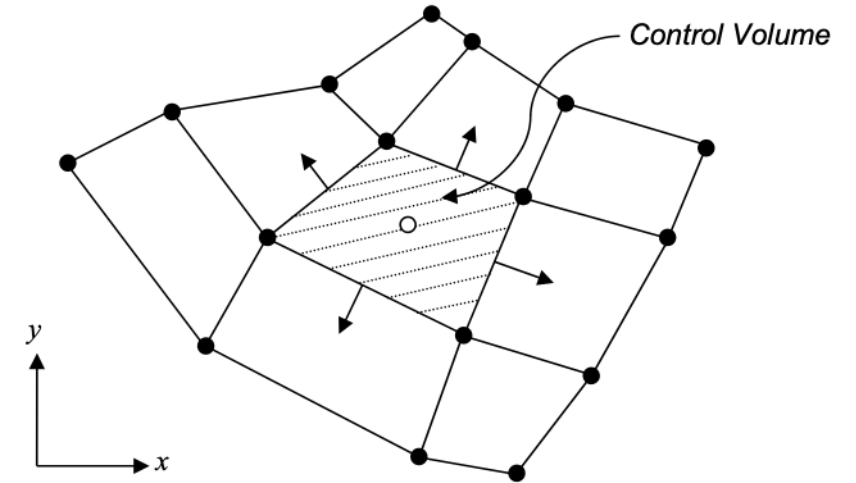
Discretization – Finite Volume

By using Gauss's divergence theorem in the x direction:

$$\int \int \int_V (\nabla F) dV = \int \int_A (F \vec{n}) dA$$

$$\frac{\partial \phi}{\partial x} = \frac{1}{\Delta V} \int_V \frac{\partial \phi}{\partial x} dV =$$

$$\frac{\partial \phi}{\partial y} = \frac{1}{\Delta V} \int_V \frac{\partial \phi}{\partial y} dV =$$

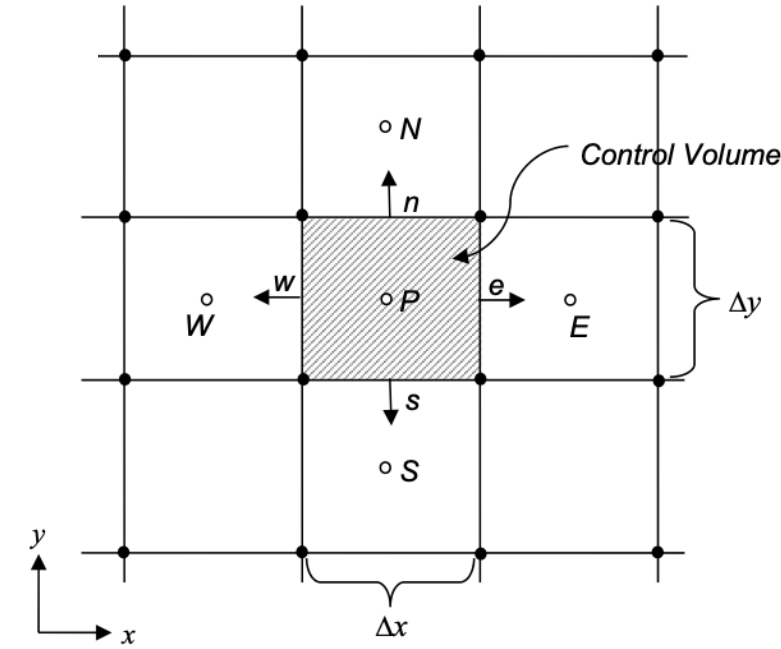


Schemes

Discretization – Finite Volume

Determine the discretized form of the 2D continuity equation using the FV method in a structured uniform grid arrangement:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$

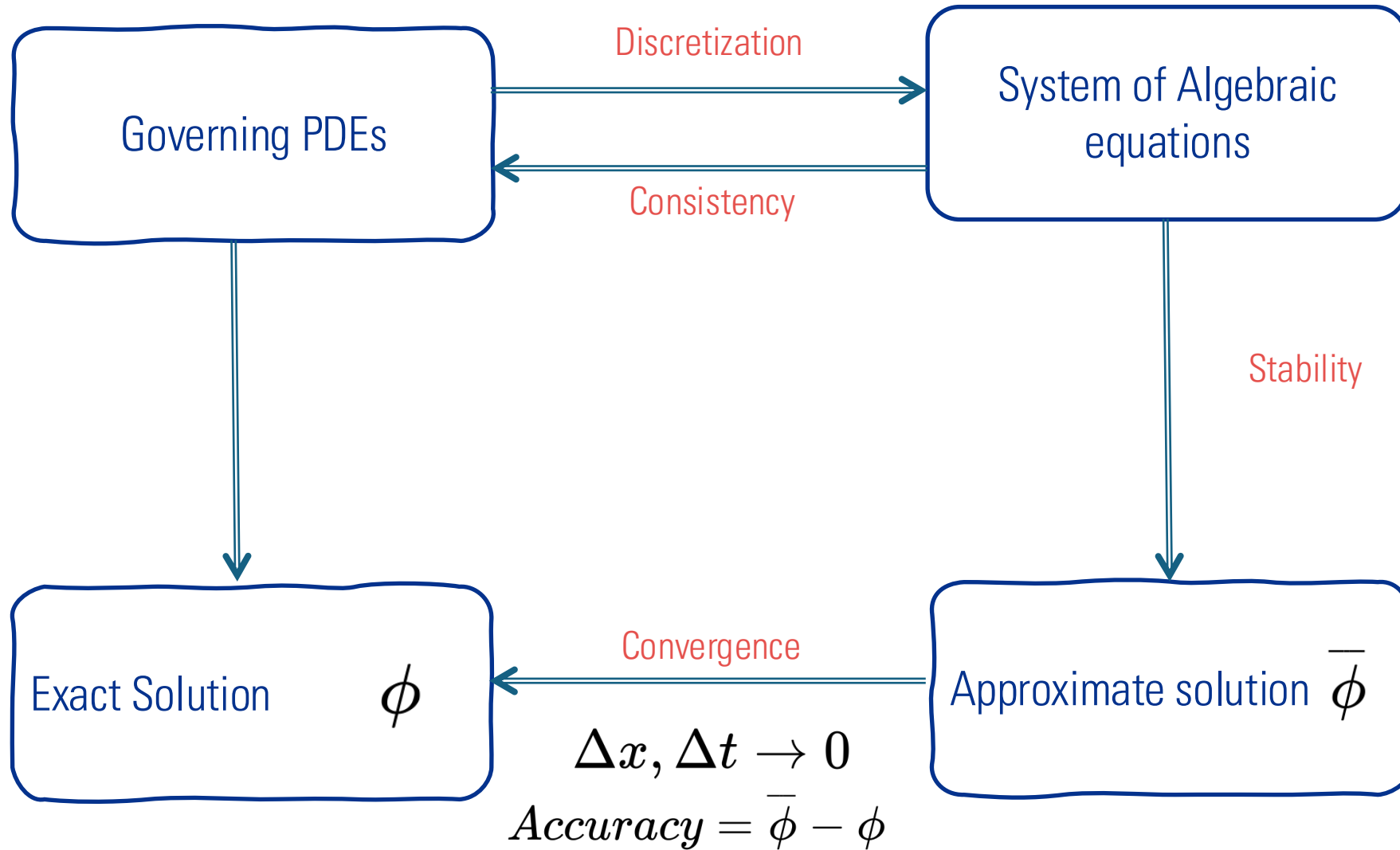


Questions so far?

Consistency, Stability, Convergence & Accuracy

Consistency, stability, convergence & accuracy

Summary



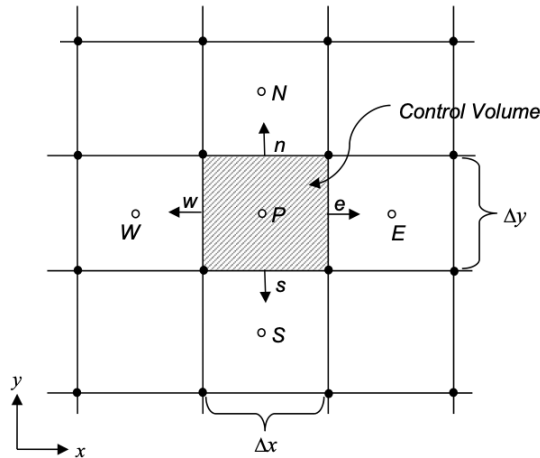
Consistency, stability, convergence & accuracy

Consistency

- For any numerical method to be consistent, the truncation error must become 0 when the time step and/or mesh spacing tend to 0
 $\Delta x, \Delta t \rightarrow 0$

- Consider the discretized form of the incompressible steady-state 2D continuity equation:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$



$$\frac{u_e - u_w}{\Delta x} + \frac{v_n - v_s}{\Delta y} = 0$$

$$u_e = \frac{u_P + u_E}{2}$$

$$\frac{u_E - u_W}{2\Delta x} + \frac{v_N - v_S}{2\Delta y} = 0$$

$$\frac{u_E - u_W}{2\Delta x} + \frac{v_N - v_S}{2\Delta y} + \underbrace{[O(\Delta x^2, \Delta y^2)]_P}_{\text{Truncation error}} = 0$$

Consistency, stability, convergence & accuracy

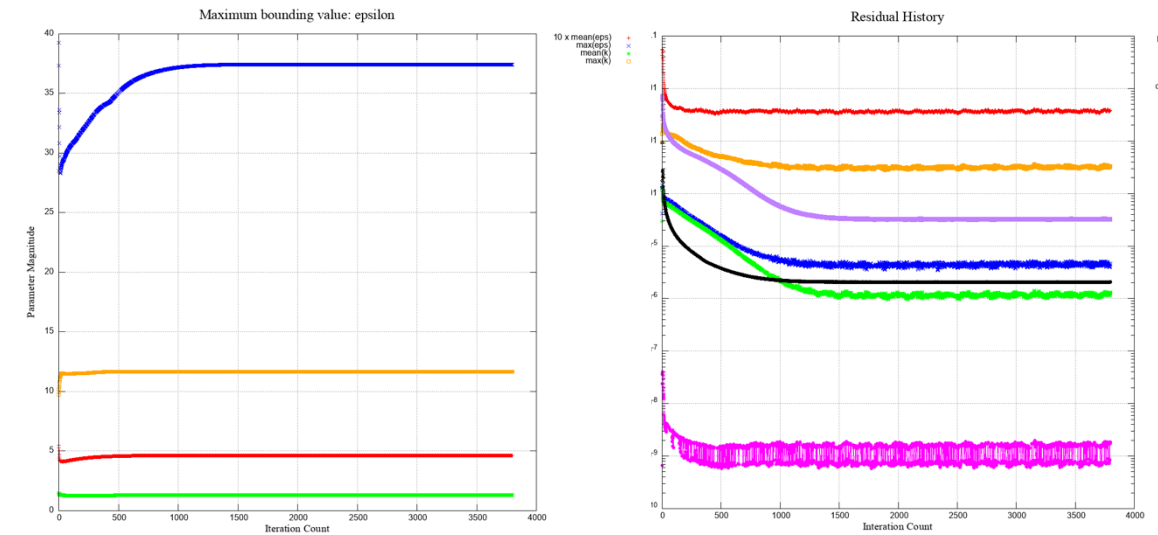
Stability

- ❑ This property concerns the growth or decay of errors introduced at any stage during the computation
- ❑ Errors = rounding off errors at every step of computation due to the finite number of significant figures the computer hardware can accommodate as well as poor initial guess
- ❑ A numerical solution method is stable if it does not magnify the errors that appear in the course of the numerical solution process
- ❑ In temporal problems guarantees that the method yields a bounded solution
- ❑ In iterative methods ensures that the solution does not diverge

Consistency, stability, convergence & accuracy

Convergence

- ❑ If a numerical method satisfies consistency and stability $\rightarrow$ we ensure convergence
- ❑ Iterative convergence:
 - ❑ All the discretized equations are deemed converged when they reach a specified tolerance at every nodal location
 - ❑ The numerical solution does not change with additional iterations
- ❑ Overall mass, momentum, scalar balances are obtained
- ❑ Residuals should diminish as the numerical process progresses
- ❑ Iterative convergence is different from grid convergence



Consistency, stability, convergence & accuracy

Accuracy

- ❑ The truncation error represents the difference between the discretized equation and the exact one, and it provides a means of evaluating the accuracy of the solution
- ❑ The order of the truncation error coincides with the order of the solution error if the grid spacings are sufficiently small and if the initial and auxiliary boundary conditions are sufficiently smooth
- ❑ Refining the grid will often produce a superior accuracy for high-order approximations over low-order approximations
- ❑ A converge solution is different from an accurate solution
- ❑ Source of errors:

Discretization

Physical modelling

Iteration/convergence

Round-off

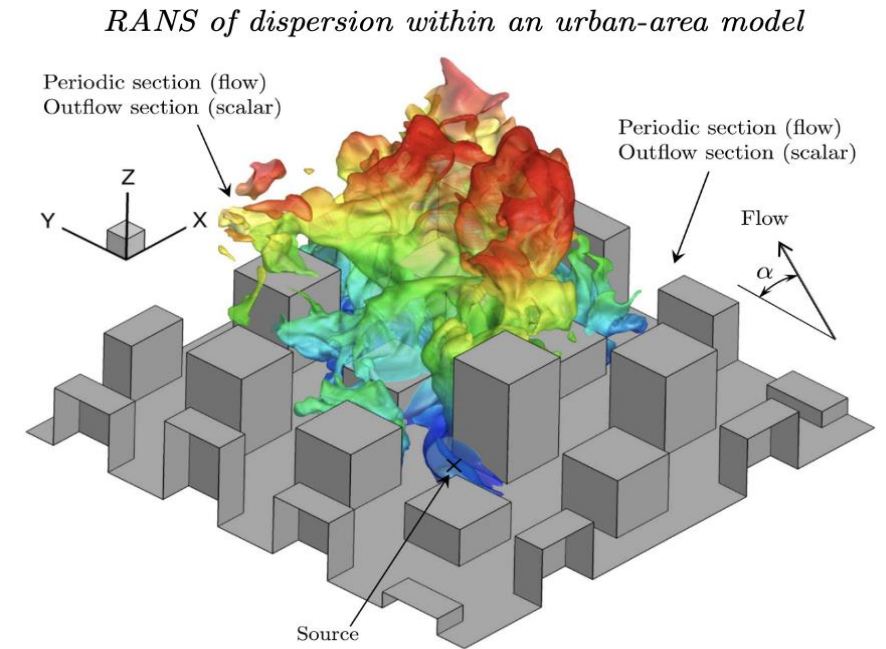
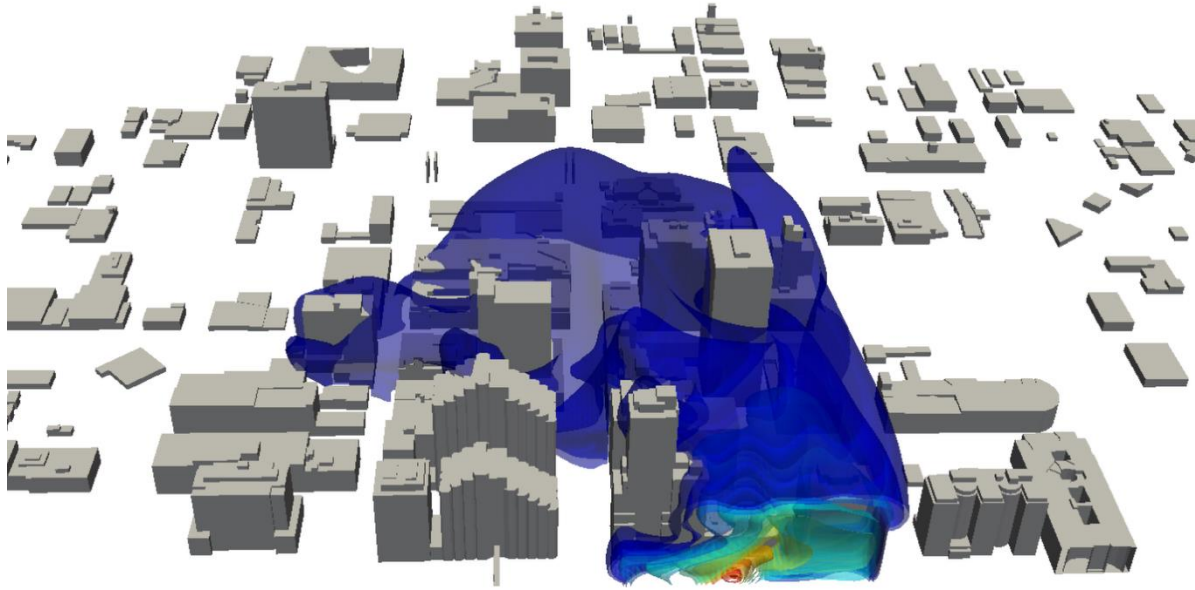
Human errors

Questions so far?

Dispersion set-up

Dispersion set-up

Introduction



[https://web.stanford.edu/group/ctr/ResBriefs/2011/16_rossi.pdf]

- ❑ Simulations of common pollutants that are non-reactive with the flow surrounding.
- ❑ Simulations of accidental spills that may happen in cities.
- ❑ Predictions of pollution that may source from industrial stacks, and their areas of influence :

Dispersion set-up

General Equations

$$\frac{\partial U_i}{\partial x_i} = 0 \quad \text{Continuity}$$

$$\frac{\partial U_i}{\partial t} + U_j \frac{\partial U_i}{\partial x_j} = -\delta_{i3}g - 2\varepsilon_{ijk}\Omega_j U_k - \frac{1}{\rho} \frac{\partial p}{\partial x_i} + \frac{1}{\rho} \frac{\partial \tau_{ij}}{\partial x_j} \quad \text{Momentum}$$

$$\frac{\partial C}{\partial t} + U_j \frac{\partial C}{\partial x_j} = \nu_C \frac{\partial^2 C}{\partial x_j^2} + Sc \quad \text{Passive scalar}$$

```
volScalarField DC("DC", turbulence->nut()/Sct+turbulence->nu()/Sc);
```

Dispersion set-up

General Equations


$$\cancel{\frac{\partial C}{\partial t}} + U_j \frac{\partial C}{\partial x_j} = \nu_C \frac{\partial^2 C}{\partial x_j^2} + Sc$$

Passive scalar

ν_C : Molecular and turbulent diffusivity

$$\nu_C = \frac{\nu}{Sc} + \frac{\nu_t}{Sc_t}$$

$$Sc_t = 0.2 - 6$$

Turbulent Schmidt number

Dispersion set-up

Implementation

What do we need to modify to make it work?

1. We need a new solver
2. We need a new equation in our solver
3. We need to iterate over the equation
4. We need to initialize the fields

Dispersion set-up

Implementation

What do we need to modify to make it work?

1. We need a new solver

```
gsclara@foote.bk.tudelft.nl ~/OpenFOAM/gsclara-7/applications/solvers/incompressible (0.111s)  
ls  
simpleCFoam  simpleTFoam  simplez0Foam
```

Dispersion set-up

Implementation

What do we need to modify to make it work?

1. We need a new solver
2. We need a new equation in our solver

```
fvScalarMatrix CEqn
(
    fvm::ddt(C)
    + fvm::div(phi, C)
    - fvm::laplacian(DC, C)
    ==
    fvOptions(C)
);

CEqn.relax();
fvOptions.constrain(CEqn);
CEqn.solve();
fvOptions.correct(C);
```

Dispersion set-up

Implementation

What do we need to modify to make it work?

1. We need a new solver
2. We need a new equation in our solver
3. We need to iterate over the equation

```
Info<< "\nStarting time loop\n" << endl;

while (simple.loop(runTime))
{
    Info<< "Time = " << runTime.timeName() << nl << endl;

    // --- Pressure-velocity SIMPLE corrector
    {
        #include "UEqn.H"
        #include "pEqn.H"
        #include "CEqn.H"
    }

    laminarTransport.correct();
    turbulence->correct();

    runTime.write();
}
```

Dispersion set-up

Implementation

What do we need to modify to make it work?

1. We need a new solver
2. We need a new equation in our solver
3. We need to iterate over the equation
4. We need to initialize the fields

```
Info<< "Reading field C\n" << endl;
volScalarField C
(
    IOobject
    (
        "C",
        runTime.timeName(),
        mesh,
        IOobject::MUST_READ,
        IOobject::AUTO_WRITE
    ),
    mesh
);

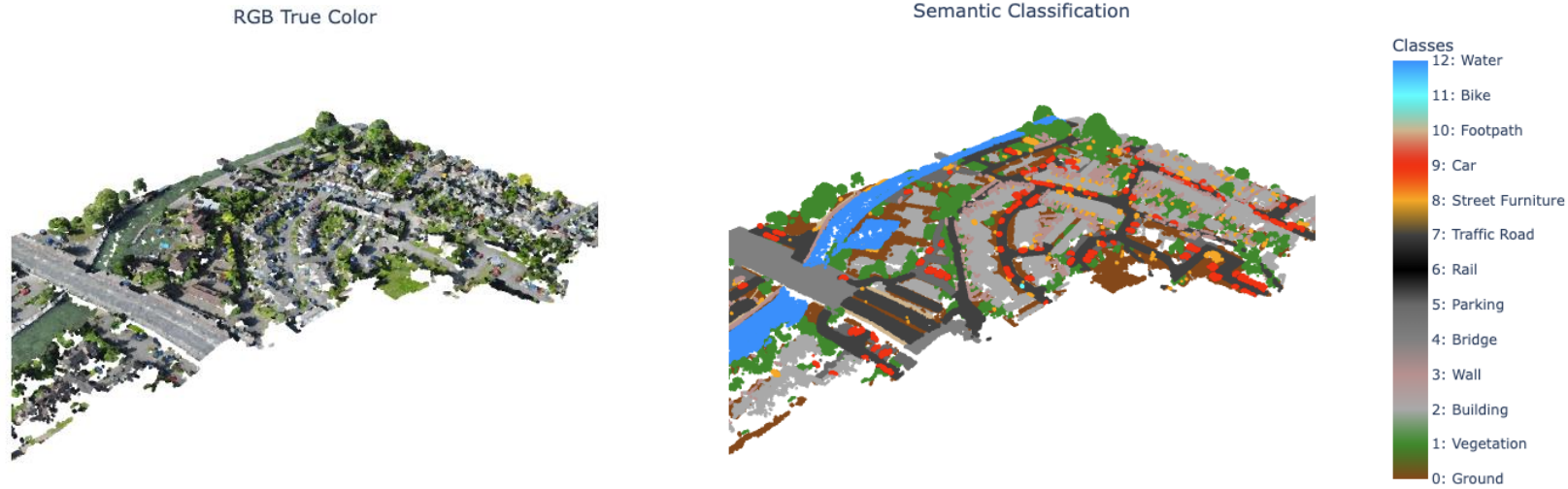
#include "createPhi.H"
```

Questions so far?

Tree set-up

Tree set-up

Introduction



Trees impact the local flows in urban environments, modifying fluxes of momentum, heat and pollution:

- ☐ They can cause deposition in their leaves
- ☐ They reduce ventilation due to their presence
- ☐ They induce evapo-transpiration within the air flow
- ☐ They provide shade when the sun is shining, changing mean radiant temperature
- ☐ Number of trees is sometimes quite large, and it is often a solution that urban planners will use to reduce heat



Tree set-up

How to model trees

Multiple level of detail on trees

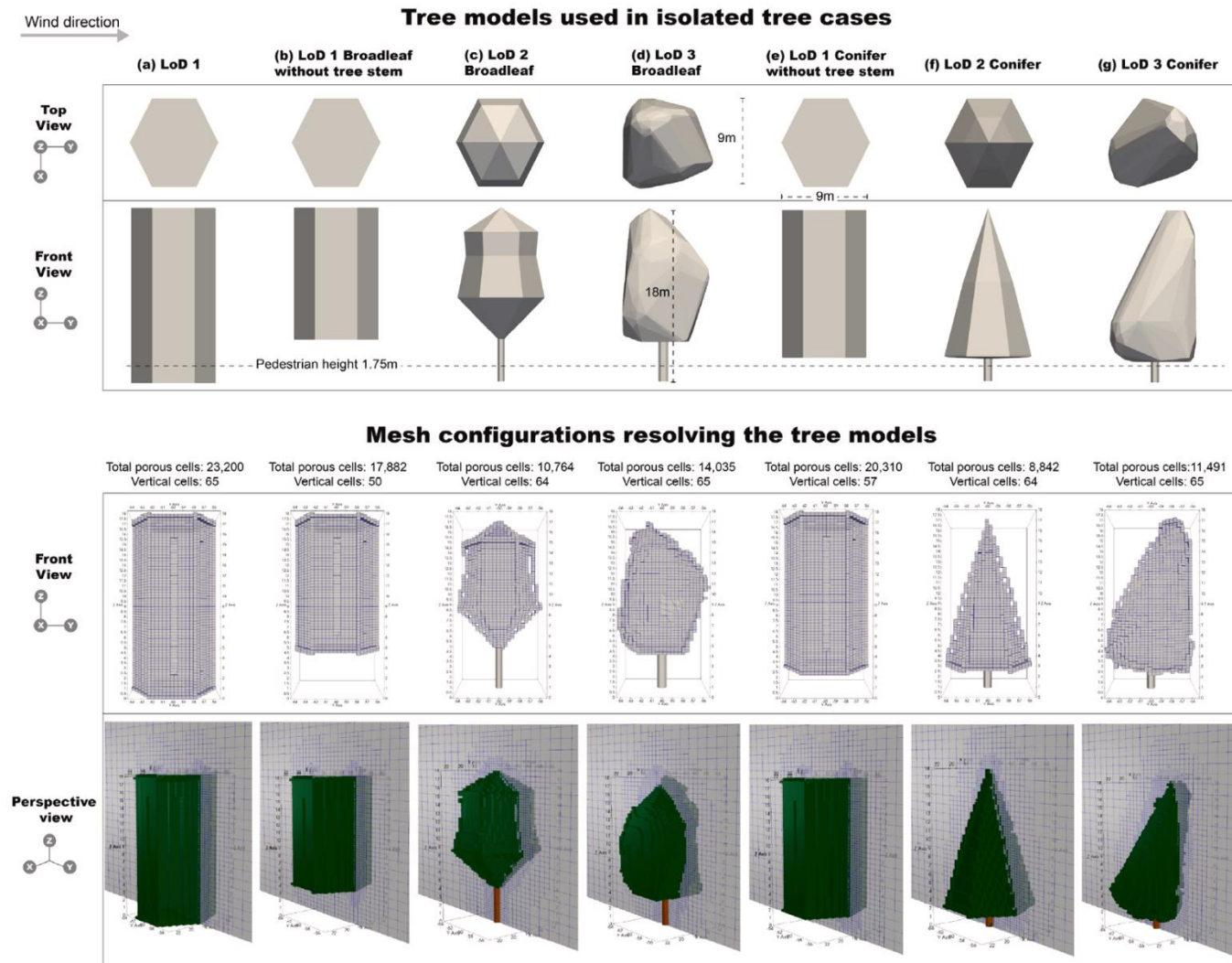


Fig. 1. The tree models and the mesh configurations used in isolated tree cases. Models (a), (c), (d), (f), (g) are generated using the tree LoD classification proposed by [de Groot \(2020\)](#). To study the effect of tree stem, we also use models (b) and (e) for the simulations. Tree stem is explicitly modeled as an obstacle while canopy is modeled as porous cells. Vertical cells refers to the number of layers across the tree models. Total porous cells refers to the number of cells resolving the tree canopy.

Fu, Runnan, Ivan Pađen, and Clara García-Sánchez. "Should we care about the level of detail in trees when running urban microscale simulations?." *Sustainable Cities and Society* 101 (2024): 105143.

Tree set-up

How to model trees

Tree porosity implementation

Momentum source:

$$S_{u_i} = -\rho C_d L A D U_i |U| \left[\frac{N}{m^3} \right]$$

Epsilon source:

$$S_\varepsilon = -\rho C_d L A D \frac{\varepsilon}{k} \left(C_{\varepsilon 4} \beta_p |U|^3 - C_{\varepsilon 5} \beta_d |U| k \right) \left[\frac{N}{m^3 s} \right]$$

Tke source:

$$S_k = \rho C_d L A D \left(\beta_p |U|^3 - \beta_d |U| k \right) \left[\frac{W}{m^3} \right]$$

- ❑ Trees' crowns are marked using a 3D geometry, and sources are imposed in those marked cells.
- ❑ Trunks are generally solid if they are available, although they are challenging to reconstruct due to the fact that point clouds are not visible due to the crown

Manickathan, L. (2019). Impact of vegetation on urban microclimate. (Doctoral Thesis), ETH Zurich, <http://dx.doi.org/10.3929/ETHZ-B-000379379>

Tree set-up

How to model trees

Tree porosity implementation

1. Momentum sink: $S_{u_i} = -\rho C_d LAD U_i |U| \left[\frac{N}{m^3} \right]$
2. Tke source: $S_k = \rho C_d LAD \left(\beta_p |U|^3 - \beta_d |U|k \right) \left[\frac{W}{m^3} \right]$
3. Epsilon source: $S_\varepsilon = \rho C_d LAD \frac{\varepsilon}{k} \left(C_{\varepsilon 4} \beta_p |U|^3 - C_{\varepsilon 5} \beta_d |U|k \right) \left[\frac{N}{m^3 s} \right]$

$|U|$ Velocity magnitude

U_i Velocity component direction i

LAD Leaf Area Density

$C_{\varepsilon 4} C_{\varepsilon 5}$ Model constants

C_d Leaf Drag coefficient

β_p Fraction of mean ke converted into tke

β_d Dimensionless coefficient for the short-circuiting of turbulent cascade

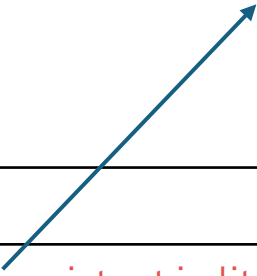
Manickathan, L. (2019). Impact of vegetation on urban microclimate. (Doctoral Thesis), ETH Zurich, <http://dx.doi.org/10.3929/ETHZ-B-000379379>

Tree set-up

How to model trees

Tree porosity implementation, typical values for constants

Depends on the tree species!!!



Variable	Range values
LAD	0.1-4, 1 (most consistent in literature)
$C_{\varepsilon 4}$	0.9-2
$C_{\varepsilon 5}$	0.9-1.8
C_d	0.2 (most consistent in literature)
β_p	1 (most consistent in literature)
β_d	4-6.5

Manickathan, L. (2019). Impact of vegetation on urban microclimate. (Doctoral Thesis), ETH Zurich, <http://dx.doi.org/10.3929/ETHZ-B-000379379>

Tree set-up

Tree effects

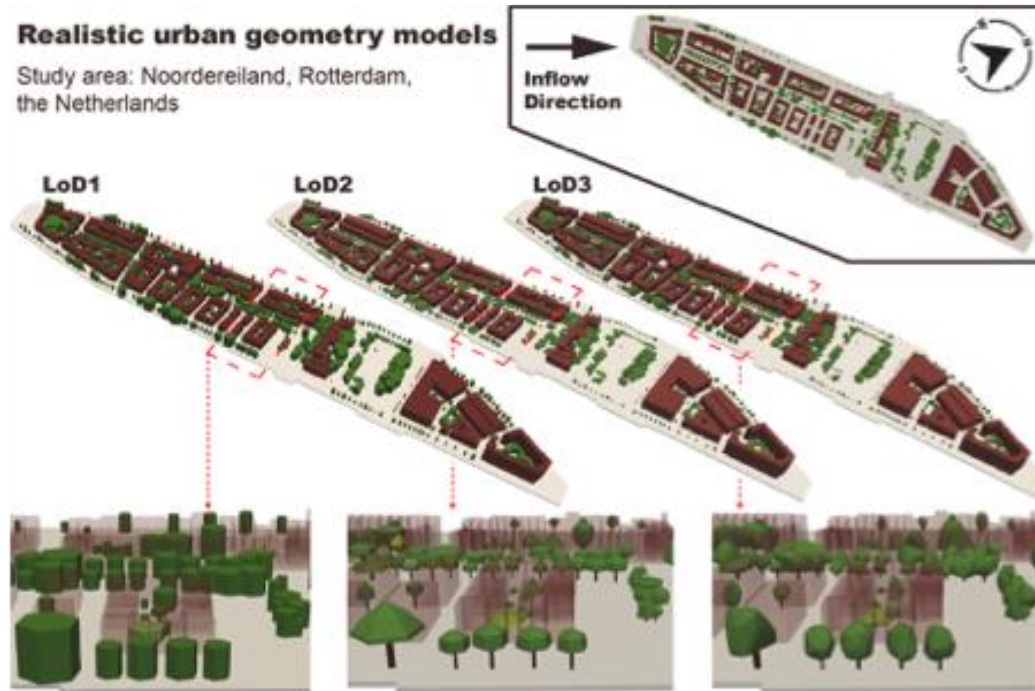
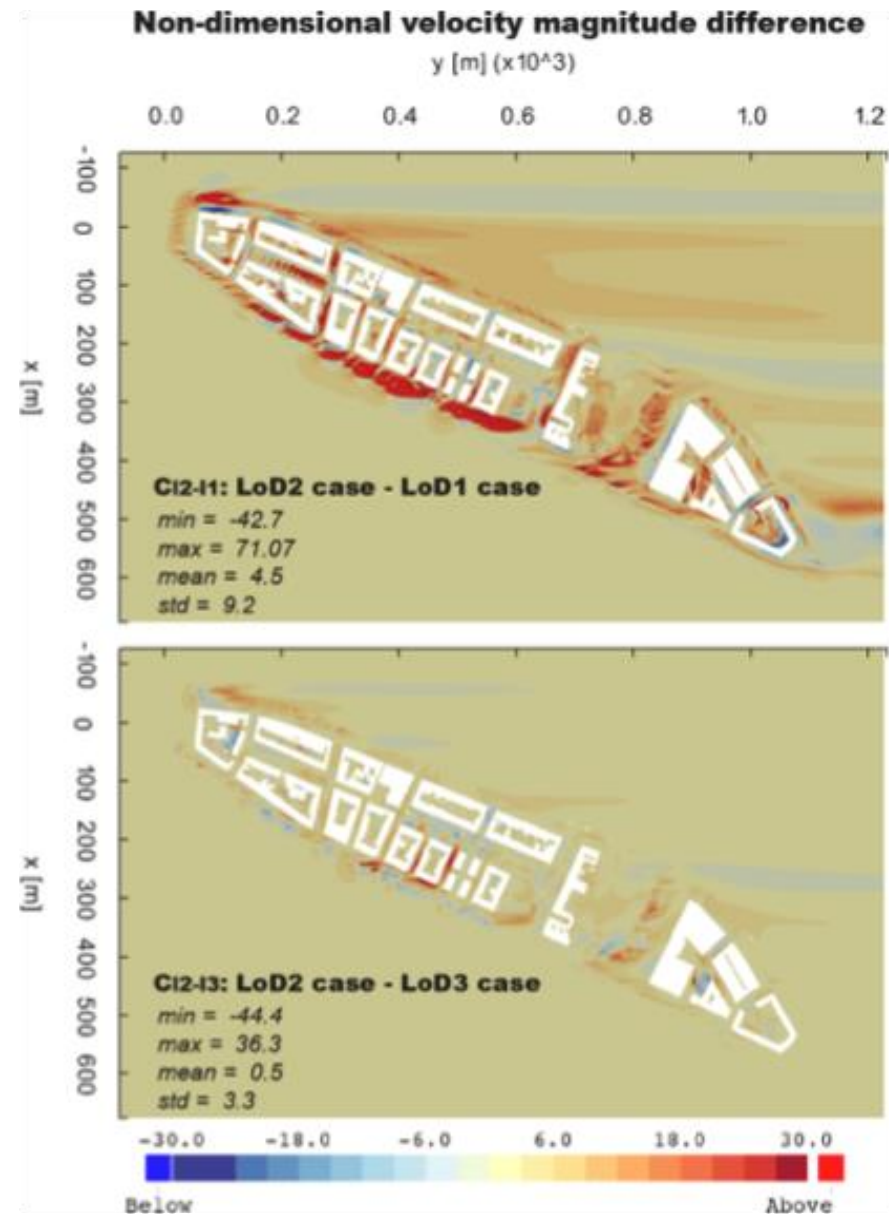


Fig. 3. Different LoD tree configurations for the realistic urban geometry model from Noordereiland in Rotterdam, The Netherlands.

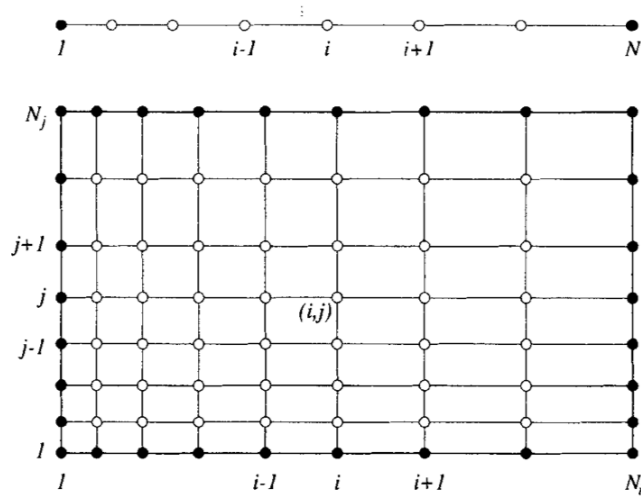


Fu, Runnan, Ivan Pađen, and Clara García-Sánchez. "Should we care about the level of detail in trees when running urban microscale simulations?." Sustainable Cities and Society 101 (2024): 105143.

Questions?

Backup

Discretization:



$$\begin{aligned} \phi_{i+1,j} &= \phi_{i,j} + \left(\frac{\partial \phi}{\partial x} \right)_{i,j} \Delta x + \left(\frac{\partial^2 \phi}{\partial x^2} \right)_{i,j} \frac{\Delta x^2}{2} + \left(\frac{\partial^3 \phi}{\partial x^3} \right)_{i,j} \frac{\Delta x^3}{6} + h.o.t. \\ \text{--- } \phi_{i-1,j} &= \phi_{i,j} - \left(\frac{\partial \phi}{\partial x} \right)_{i,j} \Delta x + \left(\frac{\partial^2 \phi}{\partial x^2} \right)_{i,j} \frac{\Delta x^2}{2} - \left(\frac{\partial^3 \phi}{\partial x^3} \right)_{i,j} \frac{\Delta x^3}{6} + h.o.t. \end{aligned}$$

$$\left(\frac{\partial \phi}{\partial x} \right) = \frac{\phi_{i+1,j} - \phi_{i-1,j}}{2\Delta x} + O(\Delta x^2)$$

Discretization

The choice influences the accuracy of the approximation

$$\phi(x) = \phi(x_i) + (x - x_i) \frac{\partial \phi}{\partial x} \Big|_i + \frac{(x - x_i)^2}{2!} \frac{\partial^2 \phi}{\partial x^2} \Big|_i + \dots + \frac{(x - x_i)^n}{n!} \frac{\partial^n \phi}{\partial x^n} \Big|_i$$

Using a forward discretisation:

$$\frac{\partial \phi}{\partial x} \Big|_i = \frac{\phi_{i+1} - \phi_i}{\Delta x} - \frac{\Delta x}{2!} \frac{\partial^2 \phi}{\partial x^2} \Big|_i + h.o.t$$

Using a central discretisation:

$$\frac{\partial \phi}{\partial x} \Big|_i = \frac{\phi_{i+1} - \phi_{i-1}}{\Delta x} - \frac{\Delta x^2}{3!} \frac{\partial^3 \phi}{\partial x^3} \Big|_i + h.o.t$$

Discretization

The central difference can be found by summing the forward and backward difference (MUDE)

Forward difference:

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{\Delta x} - \frac{\Delta x}{2!} f''(x_i) - \frac{\Delta x^2}{3!} f'''(x_i) - \dots$$

Backward difference:

$$f'(x_i) = \frac{f(x_i) - f(x_{i-1})}{\Delta x} + \frac{\Delta x}{2!} f''(x_i) - \frac{\Delta x^2}{3!} f'''(x_i) - \dots$$

Summing both terms yields:

$$2f'(x_i) = \frac{f(x_{i+1}) - f(x_i) + f(x_i) - f(x_{i-1}))}{\Delta x} - \frac{\Delta x}{2!} f''(x_i) + \frac{\Delta x}{2!} f''(x_i) + \mathcal{O}(\Delta x^2)$$

After dividing by 2 and simplifying, we get:

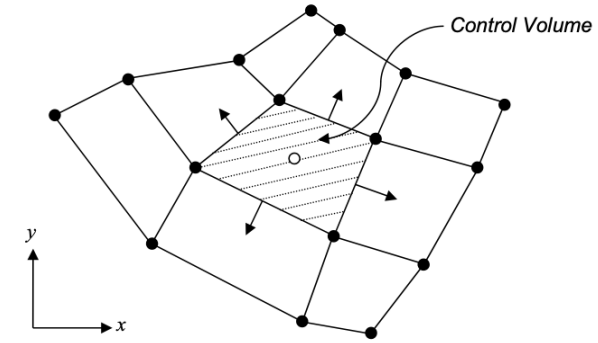
$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1}))}{2\Delta x} + \mathcal{O}(\Delta x^2)$$

Discretization

Finite Volume

By using Gauss's divergence theorem in the x direction:

$$\int \int \int_V (\nabla F) dV = \int \int_A (F \vec{n}) dA$$



$$\frac{\partial \phi}{\partial x} = \frac{1}{\Delta V} \int_V \frac{\partial \phi}{\partial x} dV = \frac{1}{\Delta V} \int_A \phi dA^x \approx \frac{1}{\Delta V} \sum_{i=1}^N \phi A_i^x$$

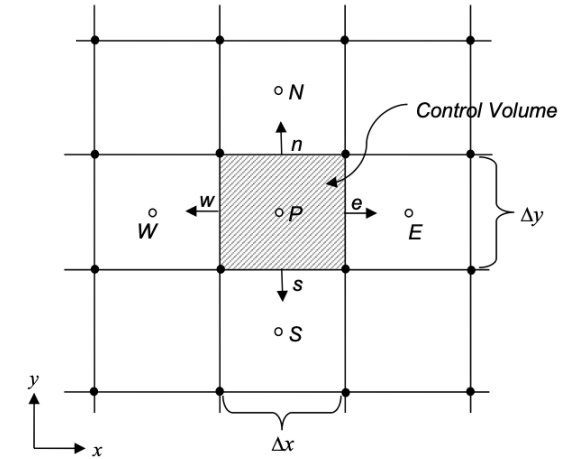
$$\frac{\partial \phi}{\partial y} = \frac{1}{\Delta V} \int_V \frac{\partial \phi}{\partial y} dV = \frac{1}{\Delta V} \int_A \phi dA^y \approx \frac{1}{\Delta V} \sum_{i=1}^N \phi A_i^y$$

Discretization

Finite Volume

Determine the discretised form of the 2D continuity equation using the FV method in a structured uniform grid arrangement:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$



$$\frac{\partial u}{\partial x} = \frac{1}{\Delta V} \int_A u dA^x \approx \frac{1}{\Delta V} \sum_{i=1}^N u_i A_i^x = \frac{1}{\Delta V} (u_e A_e^x - u_w A_w^x + \overset{0}{\cancel{u_n A_n^x}} - \overset{0}{\cancel{u_s A_s^x}}) = \frac{1}{\Delta V} (u_e A_e^x - u_w A_w^x)$$

$$\frac{\partial v}{\partial y} = \frac{1}{\Delta V} \int_A v dA^y \approx \frac{1}{\Delta V} \sum_{i=1}^N v_i A_i^y = \frac{1}{\Delta V} (\overset{0}{\cancel{v_e A_e^y}} - \overset{0}{\cancel{v_w A_w^y}} + v_n A_n^y - v_s A_s^y) = \frac{1}{\Delta V} (v_n A_n^y - v_s A_s^y)$$